CONFIDENTIAL · CANDIDATE PREP DOSSIER

Senior / Lead Full-Stack .NET & Azure

Elite 14-Day Interview Preparation Roadmap

Target: 8 YoE · Technical Lead / Principal Engineer Bar

Panel Simulation: Enterprise Architect + Engineering Director

Core Stack: C# / ASP.NET Core · React + TypeScript · SQL Server · Azure + AWS

Prepared as a strict hiring-panel coaching artifact

Contents

- 0. How To Use This Dossier
- 1. 14-Day Preparation Roadmap (Sprint Schedule)
- 2. High-Frequency Technical Questions & Senior Answers
- 3. Tech Lead Behavioral & Agile Scenarios (STAR)
- 4. Differentiating "Bonus Points" Playbook
- 5. Large-Enterprise Fit Playbook
- 6. Rapid-Fire Closing Round & Questions To Ask Them
- 7. One-Page Cheat Sheet (print & carry)

0. How To Use This Dossier

Not a glossary — a judgment trainer. Every answer is written as an engineer who has owned production incidents and led squads. Three rules:

- **Calibrate, don't memorize.** Rehearse the trade-off reasoning, not the wording.
- **Close every loop.** End each technical answer with the trade-off you accepted and how you'd *measure* it was right (p99, error budget, cost, MTTR).
- **Decision first, then justify, then the escape hatch** ("where this breaks down is...").

> **Signal they're buying:** they stop asking "how" and start asking "when would you *not* do this?" — that's the seniority probe. Welcome it.

1. 14-Day Preparation Roadmap (Sprint Schedule)

Two weeks = one sprint; the panel is the demo. Each day has a **Build** (hands-on), **Drill** (recall under pressure), **Reflect** (one-paragraph trade-off).

Phase 1 — Days 1–4: Core Backend Depth

Day 1 — .NET Runtime Internals & Async Model

- **Build:** Minimal API exercising `async/await`, `ValueTask`, `IAsyncEnumerable`; watch ThreadPool queue length with `dotnet-counters` under load.
- **Study:** Task vs ValueTask; no sync-context in Core; thread-pool starvation from sync-over-async.
- **Drill:** Trace one HTTP request, Kestrel socket → response flush. **Reflect:** When is `ValueTask` a net negative?

Day 2 — Memory, GC & Allocation Discipline

- **Build:** Profile a hot path; convert a string loop to `Span<T>`; measure Gen0 before/after.
- **Study:** Generational GC, LOH (>85KB) fragmentation, Server vs Workstation GC, `ArrayPool<T>`, `stackalloc`.
- **Drill:** "p99 spikes every ~30s" → walk the GC hypothesis.

Day 3 — EF Core vs Dapper

- **Build:** Same query both ways (tracking vs `AsNoTracking`, split vs single, compiled); compare generated SQL + plans.
- **Study:** N+1, cartesian explosion, change-tracker cost, `DbContext` pooling. **Drill:** Defend a hybrid (EF writes, Dapper read-models).

Day 4 — SQL Server Performance & Concurrency

- **Build:** Reproduce a deadlock, capture the graph via Extended Events, fix with access ordering + covering index.
- **Study:** Clustered vs non-clustered, composite key order, included columns, parameter sniffing, RCSI, `rowversion`.
- **Drill:** Read a plan aloud — key lookup, seek vs scan, spill warnings.

Phase 2 — Days 5–7: Frontend & System Architecture

Day 5 — React Performance & State at Scale

- **Build:** Profile a re-render cascade; fix with `memo`, stable callbacks, selectors, virtualization.
- **Study:** RTK vs Context vs server-state (React Query); referential equality; reconciliation. **Drill:** "One input re-renders the dashboard" — diagnose in 90s.

Day 6 — API & Front-End Architecture

- **Build:** Design a BFF; REST vs GraphQL for an aggregation screen; a micro-frontend seam + failure modes.
- **Study:** token storage (`httpOnly` +CSRF vs localStorage+XSS), idempotency keys, WebSockets vs SSE vs polling.

Day 7 — Cloud Patterns & Zero Trust

- **Build:** Whiteboard an event-driven multi-tenant pipeline on Azure (Service Bus + Functions + Key Vault + Front Door) + AWS twin.
- **Study:** managed identity, Zero Trust triad, OAuth2/OIDC, private endpoints, Blue/Green + Canary. **Reflect:** pool vs silo vs bridge tenancy.

Phase 3 — Days 8–11: Technical Leadership & Delivery

Day 8 — Agile Execution & Capacity Mapping

- Velocity vs commitment, capacity math (PTO/ceremonies/support), skill-to-story mapping, ~20% debt reserve. Draft the scope-creep STAR.

Day 9 — Stakeholder & Delivery-Risk Management

- Escalation framing (impact/options/recommendation/ask), status reporting that shields without hiding, RAID logs. Draft the release-block STAR.

Day 10 — Code Quality, Architecture & Mentoring

- Clean Architecture/SOLID without dogma, PR governance, the "brilliant jerk," lifting an underperformer with a concrete plan.

Day 11 — CI/CD Governance & Test Strategy

- Trunk-based vs GitFlow, PR gates, DORA metrics, test-pyramid ownership, automated rollback policy, environment strategy.

Phase 4 — Days 12–14: AI Tooling, Live Design & Mock Loops

Day 12 — AI-Assisted Development Narrative

- 3-minute story: Composer multi-file refactor, project `.cursorrules`, AI-generated tests you reviewed, a diff you *rejected* for security/memory.

Day 13 — Live System Design Drill (timed)

- 45 min: "Multi-tenant transaction system, 10k TPS, exactly-once effect." Record; grade on framing, trade-offs, failure modes, capacity math.

Day 14 — Executive Mock Loops & Reset

- Two mocks (technical-deep + leadership). Then stop — light review of STAR cards only. Sleep > cramming.

2. High-Frequency Technical Questions & Senior Answers

2.1 C# / ASP.NET Core Web API

ASYNC LIFECYCLE

Q1. What actually happens when an `async` action awaits a DB call under load, and why does it scale?

At the await, the method becomes a state machine; if the I/O isn't done, the continuation is registered and **the thread returns to the pool** rather than blocking. IOCP signals completion, the continuation resumes on a pool thread. This scales because threads are expensive (~1MB stack + context switches): a handful service thousands of I/O-bound requests. ASP.NET Core has **no sync-context**, so `ConfigureAwait(false)` is a no-op in app code (still matters in libraries).

Breaks down: nothing gained for CPU-bound work. The classic killer is *sync-over-async* (`.Result` / `.Wait()`): a blocked pool thread triggers **thread-pool starvation** (pool injects ~1 thread/500ms), latency climbs while CPU sits idle. I've root-caused exactly that — fix was async end-to-end, not more threads.

VALUETASK

Q2. When `ValueTask<T>` over `Task<T>`, and when is it a trap?

`Task<T>` heap-allocates per call. On a hot path that usually completes synchronously (cache hit, buffered read), `ValueTask` allocates nothing. Traps I treat as rules: await it *exactly once*, never twice, never read `.Result` before completion, never store-and-await-later — if I need any of those, call `.AsTask()` once. Default to `Task` on public APIs; reach for `ValueTask` only when a profiler proves the allocation matters on a genuinely hot, sync-completing path (it can be slower if usually async).

GC / LOH

Q3. Periodic latency spikes + rising memory. Pin it on GC and pull the levers.

Confirm with data: `dotnet-counters` for heap size, Gen0/1/2, % time in GC, alloc rate; if spikes track Gen2/LOH, that's the smoking gun. Then a trace to find *who* allocates. Usual causes/fixes: **LOH fragmentation** (objects >85KB — big byte[]/serialization) → pool with `ArrayPool<T>`/RecyclableMemoryStream, stream not buffer; **Gen0 churn** (LINQ/string/boxing hot loops) → `Span<T>` / `stackalloc` ; **wrong GC mode** → Server + concurrent GC for throughput; **mid-life leaks** (static handlers, unbounded caches) → weak refs, bounded `IMemoryCache` . **Trade-off:** Server GC costs memory + background threads; on a constrained container validate against the limit to avoid OOMKills.

MIDDLEWARE

Q4. Design a middleware pipeline for correlation IDs, exception handling, tenant resolution. Ordering pitfalls?

Order *is* behavior. Mine: exception/problem-details first (wraps all) → correlation-id enrichment → HTTPS/forwarded-headers → authN → authZ → tenant resolution (needs principal) → routing/endpoint. Anything needing endpoint metadata goes after `UseRouting` ; inject per-request services via `InvokeAsync` params, not the constructor (middleware is singleton). **Burned by:** setting headers after the response started (use an `OnStarting` callback instead) and capturing a scoped `DbContext` in the singleton. Tenant resolution writes to a scoped `ITenantContext` so EF query filters auto-scope.

2.2 SQL Server & Data Engineering

QUERY LATENCY

Q5. Fast-before, slow-now, same query, prod only. Diagnostic path?

Don't rewrite before I read the plan. Capture the **actual** plan + Query Store stats; look for seek vs scan, an expensive **key lookup** at volume, estimated-vs-actual row skew (stale stats), spills (under-sized grant), and two plans for one query (**parameter sniffing**). "Fast then slow" is almost always: stale statistics → update/force plan; parameter sniffing → `OPTIMIZE FOR` / `RECOMPILE`; blocking → check waits in `dm_exec_requests`; or a missing covering index as the table grew. Query Store's regression view shows the exact moment + plan that changed — I bring that artifact, not a guess.

DEADLOCKS

Q6. Recurring deadlocks under heavy writes — without just retrying.

Retries are a seatbelt. Capture the **deadlock graph** (XE `xml_deadlock_report`). Root causes: (1) inconsistent lock ordering → enforce a canonical access order everywhere; (2) wide scans taking too many locks → covering/narrower index so writers seek; (3) long transactions → shrink scope. *Then* add idempotent retry-on-1205 with jittered backoff as defense-in-depth, and consider **RCSI** to take readers off writers — naming the trade-off (tempdb pressure, changed read semantics), so I validate tempdb first.

INDEXING

Q7. Clustered vs non-clustered; composite column order?

Clustered = physical order; one per table; narrow, static, ever-increasing (a random GUID clustered key causes page splits — use sequential key, keep GUID as non-clustered unique). Non-clustered are B-trees pointing at the clustered key. Composite order = **Equality, Range, then Include**: equality/most-selective columns lead, range next, select-only columns into `INCLUDE` to make it **covering** (kills key lookups). (A,B) can seek on A or A+B, not B alone. Every index costs on every write — I audit `dm_db_index_usage_stats` for unused/duplicate rather than carpet-bombing.

CONCURRENCY

Q8. Optimistic vs pessimistic under heavy traffic?

Default **optimistic** for read-heavy web contention: a `rowversion` in the UPDATE's WHERE; zero rows → someone beat me → surface a conflict (EF's `DbUpdateConcurrencyException`). No locks across think-time, so it scales. **Pessimistic** (`UPDLOCK` / `HOLDLOCK`) only for short, high-value critical sections where a conflict is costly to reconcile (scarce inventory, unique allocation) — keep the locked region tiny. For money movement I add an **idempotency key** so a retry can't double-apply — concurrency and idempotency are separate problems, both handled.

2.3 React.js & Full-Stack Integration

STATE AT SCALE

Q9. Structuring state in a large React app for performance + maintainability?

Biggest lever: **separate server state from client state**. Fetched data → React Query/RTK Query (caching, dedup, background refetch, stale-while-revalidate) — deletes mountains of manual loading/error boilerplate. True client state (auth, flags, theme) → RTK/Zustand with selectors. Ephemeral UI → local `useState`. Use **Context sparingly** — it's DI, not a store; every consumer re-renders on change, so split providers and keep it low-frequency. Discipline that pays: colocation + selector reads.

RENDERING

Q10. Dashboard re-renders excessively. Find and fix the cascade.

React DevTools Profiler shows which components render and *why*. Usual culprits: new object/array/fn literals passed as props (breaks referential equality), state lifted too high, unstable Context value. Surgical fixes: `React.memo` on pure leaves; `useCallback` / `useMemo` to stabilize *references* only where a memoized child/effect needs it; move state down or use a selector store; **virtualize long lists** (`react-window`). I avoid cargo-cult `useMemo` — it has cost and masks bad data flow — and re-profile to verify commit counts dropped.

TOKEN SECURITY

Q11. localStorage or cookies for the SPA token? Defend it.

`Secure`, `HttpOnly`, `SameSite` **cookie**, not localStorage. localStorage is JS-readable, so one **XSS** exfiltrates the token instantly; `HttpOnly` is invisible to JS. Cookie cost is **CSRF** — mitigated with `SameSite` + anti-forgery tokens + tight CORS. What I ship is a **BFF**: SPA holds an `HttpOnly` session cookie; the backend holds the real OAuth tokens and calls downstream — the browser never sees a bearer token. Plus short-lived access + refresh rotation and a strict CSP. "No XSS" is the real goal; cookie storage just caps the blast radius.

REALTIME

Q12. Polling vs SSE vs WebSockets — choose and make it resilient.

By data shape, not trend. **Polling** (long-poll / interval + conditional `ETag`/304) for low-frequency, tolerant updates — operationally boring, a virtue. **SSE** for one-way server→client streams (notifications, progress). **WebSockets**/SignalR for low-latency bidirectional (chat, collab). Resilience: plan for the socket *failing* — backoff+jitter reconnect, heartbeat for half-open, sequence numbers to resync missed events, fallback to SSE/long-poll (SignalR negotiates). At scale name the scale-out problem: sticky sessions or a **backplane** (Redis/Azure SignalR) so a message on one instance reaches clients on another.

2.4 Cloud Architecture (Azure & AWS)

SYSTEM DESIGN · FLAGSHIP

Q13. Design an event-driven, multi-tenant transaction system. ~10k TPS, exactly-once effect, zero-downtime deploys. Azure, with the AWS twin.

1. Clarify: TPS/burst, payload, latency SLO, ordering (per-account vs global), tenant count/isolation/compliance, and what "exactly-once" means — I deliver *effectively-once* via idempotency (true exactly-once *delivery* is a myth).

2. Edge: Front Door (global LB + WAF + TLS) → **API Management** (throttling, per-tenant rate limits, JWT validation). Intake API is thin: validate, auth tenant, assign **idempotency key**, publish — no inline work.

3. Messaging: Service Bus for commands (FIFO via *sessions* keyed on account id, DLQ, dedup, competing consumers); **Event Hubs** for high-volume event fan-out (audit/analytics/projections).

4. Compute: Functions (or Container Apps/AKS) as competing consumers scaling on queue depth (KEDA). Processing is **idempotent**: check the key against a dedup store before applying — a redelivery (Service Bus is at-least-once) is a no-op. That's effectively-once.

5. Tenancy & storage: present the spectrum — *silo* (DB/tenant: strongest isolation, costlier), *pool* (shared schema + RLS: cheapest density, noisy-neighbor risk), *bridge*. Typical: pool + row-level security for the long tail, silo for compliance-bound enterprise tenants — a pragmatic hybrid. Azure SQL is the ACID system of record.

6. Consistency: transactional outbox so DB write + "event published" commit atomically; a relay publishes. Cross-service flows use a **saga** with compensations, not 2PC.

7. Secrets/identity: Key Vault via **managed identity** — no secrets in config. Zero Trust throughout.

8. Zero-downtime: Blue/Green via deployment slots (warm green, smoke test, swap) + **Canary** (Front Door weighted 5→25→100%) gated on live SLIs with auto-rollback. Schema changes are **expand/contract** so old+new run concurrently.

9. Observability/resilience: App Insights + OpenTelemetry tracing across async hops (correlation id rides the message), DLQ alerts, Polly retries/circuit-breakers. Alert on error-budget burn, not raw counts.

Concern	Azure	AWS
Edge / CDN / WAF	Front Door	CloudFront + WAF
API gateway	API Management	API Gateway
Ordered commands	Service Bus (sessions)	SQS FIFO
Event stream	Event Hubs	Kinesis / MSK
Serverless	Functions	Lambda
Containers	Container Apps / AKS	ECS Fargate / EKS
Relational	Azure SQL	RDS / Aurora
Secrets	Key Vault + MI	Secrets Manager + IAM
Observability	App Insights / Monitor	CloudWatch / X-Ray

Close with: "At-least-once delivery + idempotent consumers + transactional outbox gives exactly-once *effect* without pretending exactly-once *delivery* exists."

ZERO TRUST

Q14. What is Zero Trust in that system — concretely?

Three principles, each a control. **Verify explicitly:** every service call authenticated via **managed identity** + Entra ID, JWTs validated (sig/issuer/audience/expiry) at APIM *and* the service. **Least privilege:** scoped RBAC (this function reads *this* secret, writes *this* queue, nothing else) + row-level security per tenant. **Assume breach:** private endpoints (stores not public), Key Vault with rotation, encryption in transit/at rest, full audit logging. The shift: the network is not the trust boundary — identity is.

3. Tech Lead Behavioral & Agile Scenarios (STAR)

~2 minutes each. Always land the **result with a metric** + a one-line "what I'd do differently." Self-awareness scores as high as the win.

3.1 Resource Mapping & Scope Creep

Situation: 6-engineer squad, committed quarterly roadmap, a payments rewrite carrying real tech debt. Week 2, a major client escalated via our VP demanding a net-new reporting feature "by month end."

Task: Own both the commitment and codebase health — satisfy a revenue-impacting stakeholder without torching capacity or letting debt become an outage.

Action: No blind yes/no. Ran a capacity map (velocity minus support + PTO), decomposed the ask with two seniors into a thin vertical slice (20% delivering 80% of the need), and gave the VP three options with trade-offs: (A) full feature, payments slips 3 wks; (B) thin slice in 2 sprints, milestone intact; (C) contractor, highest cost. Quantified the debt as reliability risk (recent incident rate). Chose B; protected ~20% capacity for debt; tracked both streams on one board.

Result: Slice shipped in 11 days; client renewed; payments milestone held; debt work cut the module's incident rate ~60% that quarter. *Differently:* socialize a lightweight intake/change process earlier to avoid the fire-drill triage.

3.2 Delivery Bottleneck & Status Reporting

Situation: Two days before a contractual release, integration testing surfaced a data-corruption edge case in a downstream integration we didn't own. Management was already briefing the client on the date.

Task: Decide ship/hold/partial and communicate up honestly while keeping engineers on the fix, not the politics.

Action: Reproduced it, scoped blast radius (which tenants, reversibility), confirmed a correctness issue — hard no-ship on a payments path. Took status reporting onto myself; gave the team one focused objective. To management: led with impact ("silent corruption risk for ~3% of tenants"), options (full hold vs feature-flagged release to unaffected tenants), a recommendation (flagged partial), a concrete ask (one day + 9am go/no-go), and a scheduled update cadence so they stopped pinging engineers.

Result: Flagged release to unaffected tenants on date, risky path held one day, fixed with the partner. Zero corruption incidents, relationship intact, leadership praised the transparency. *Lesson:* execs handle bad news — *surprise* erodes trust, so report risk early with options attached.

3.3 Architecture Conflict, Code Quality & Mentoring

Situation: A strong, opinionated senior kept merging large PRs that bypassed Clean Architecture boundaries (logic in controllers, data access from the API layer); meanwhile a junior was stalling — PRs sitting for days, under-confident.

Task: Protect architectural integrity without a personality war, and unblock + grow the struggling engineer.

Action: Moved the debate out of PR comments into a 30-min design session, reframed around agreed principles (testability, change isolation) not my preference. Codified boundaries as an **ADR** and enforced them in CI with an architecture test (NetArchTest) + PR templates — the rule became the system's, not mine; he became an advocate. For the junior: paired him with a senior, broke work into small PRs, gave him a low-risk but visible component to own, weekly 1:1s on confidence + specific feedback.

Result: Violations dropped to ~zero (CI caught them pre-review); review time fell. The junior shipped his component in six weeks and was taking stories independently next quarter. *Differently:* establish ADR + architecture tests at kickoff — encoding conventions up front beats fixing drift.

4. Differentiating "Bonus Points" Playbook

4.1 Cursor AI IDE in Production — Force Multiplier, Not Crutch

Framing that lands: **"AI accelerates the parts I already know how to verify; it never makes the engineering decision."**

- **Composer multi-file refactors:** "Renaming a bounded context across ~30 files — DTOs, handlers, tests — I drove with Composer and reviewed the diff as one unit: an afternoon → ~20 min, and I read every hunk." Human owns intent + review.
- **Project `.cursorrules`:** "I encode our conventions — Clean Architecture boundaries, async-all-the-way, no `.Result`, logging/error patterns — so output is born compliant and consistent across the squad. Executable onboarding docs."
- **AI-assisted tests:** "It writes the boring arrange/act + obvious edges; I hand-write the tricky concurrency/boundary tests. I review assertions, not green checkmarks — a test that passes but asserts wrong is worse than none."
- **Auditing AI diffs (the maturity signal):** "I hunt its failure modes — missing `using`, leaky closures, sync-over-async, string-interpolated SQL, hardcoded secrets, over-broad catch. AI is fluent and confident, which is exactly why it needs a skeptical senior reading the diff."

> **Trap:** never imply you ship AI output unreviewed or "couldn't do it without AI." Posture: "It's a power tool. I'm accountable for every line that merges under my name."

4.2 Testing Strategy & SRE

Test pyramid I enforce

- **Unit (base):** xUnit/NUnit + Moq. Fast, deterministic, behavior not implementation; gate every PR.
- **Integration (high-value middle): Testcontainers** spins real SQL Server/Redis/Service Bus per run — catches what mocks hide (real SQL, transactions, migrations) without shared-env flakiness. I push hardest here.
- **E2E (thin top): Playwright** on a few critical journeys (login, checkout, money path). Deliberately thin — slow/flaky at volume.

SRE & observability

- **SLIs/SLOs & error budgets:** SLO (e.g. 99.9% <300ms) managed to a budget — healthy → ship fast; burning → freeze features, spend on reliability. Turns "stable enough?" from argument into policy.
- **OpenTelemetry / App Insights:** tracing with a correlation id surviving async/queue hops, structured logs, RED/USE metrics — goal is cutting **MTTR** (alert → root cause in minutes via a trace).
- **CI/CD + auto-rollback:** trunk-based, PR gates (build/tests/SAST/coverage), progressive delivery, **auto-rollback** on SLI breach, **DORA metrics** as the delivery-system dashboard.

5. Large-Enterprise Fit Playbook

At a large enterprise the panel weighs **blast-radius thinking, governance, and cross-team influence** as heavily as raw coding. Calibrate your answers to these realities.

5.1 What the enterprise panel is really testing

Signal	What they listen for
Operating at scale	You think in percentiles, error budgets, and cost — not "it works on my machine." Capacity math and failure modes are reflexive.
Governance maturity	Change management, approvals, audit trails, segregation of duties, and compliance (SOC 2 / ISO 27001 / PCI / HIPAA as relevant) are table stakes, not friction you resent.
Cross-team influence	You ship through other teams — platform, security, DBAs, release management — via ADRs, RFCs, and clear written proposals, not heroics.
Risk & reversibility	Every change has a rollback, a feature flag, and a blast-radius assessment. You prefer boring, reversible steps over clever big-bangs.
Legacy pragmatism	You modernize via strangler-fig and incremental migration, respecting systems that make money, rather than demanding rewrites.

5.2 Enterprise-flavored talking points

- **Brownfield over greenfield:** "I assume there's a system of record I must integrate with and can't break. I reach for the *strangler-fig* pattern — put a facade in front, route slices to the new service, decommission the old path once parity + metrics prove out."
- **Platform vs product boundaries:** "I distinguish shared-platform concerns (auth, observability, CI templates, paved roads) from product-squad ownership, and I consume the golden path rather than reinventing it — and I push back when the platform blocks delivery, with data."
- **Security & compliance as design input:** "Threat modeling, data classification, and least-privilege are inputs at design time, not an audit scramble before go-live. Secrets in Key Vault, managed identity, private endpoints, and audit logging are defaults."
- **Release governance:** "Change-advisory expectations, maintenance windows, and staged rollouts are part of my design, and I automate the evidence (deployment records, test results, approvals) so governance doesn't throttle throughput."
- **Vendor & cost awareness:** "I weigh build-vs-buy and FinOps — reserved capacity, autoscale floors, data-egress cost — because at enterprise scale an architecture choice is a budget line."

5.3 Enterprise curveballs & one-line stances

Question	Senior stance
"Our monolith is slowing us down — microservices?"	Only if teams/deploys are actually coupled. First a modular monolith with clean boundaries; split a service when a bounded context needs independent scale/deploy — never services sharing one DB.
"How do you migrate a legacy system with zero downtime?"	Strangler-fig + expand/contract schema + dual-write/shadow-read with reconciliation, cut over behind a flag, keep rollback until metrics prove parity.
"A critical third-party API is flaky."	Polly: timeout + retry-with-jitter + circuit breaker + bulkhead; cache/fallback where staleness is acceptable; queue and degrade gracefully rather than cascade-fail.
"Security wants to block your release."	Treat them as a stakeholder, not a gate: understand the finding, quantify risk, propose compensating controls + a remediation timeline, decide with data — never route around them.
"How do you handle a multi-region outage?"	Pre-defined RTO/RPO, active-passive or active-active with health-probe failover (Front Door/Traffic Manager), tested runbooks, and a blameless postmortem that produces action items, not blame.

6. Rapid-Fire Closing Round & Questions To Ask Them

Rapid-fire self-check (<30s each)

Prompt	One-line senior answer
`IEnumerable` vs `IQueryable` at a repo boundary?	Return `IEnumerable`; `IQueryable` leaks the provider and risks client-eval / open-ended downstream queries.
Why not `async void`?	Exceptions can't be awaited/caught — they crash the process. Only valid for event handlers.
`record` vs `class`?	Records for immutable value-semantics DTOs/messages; classes for entities with identity + behavior.
CTE vs temp table vs table variable?	CTE for readability (not perf); temp table when the optimizer needs stats on a big intermediate; table variable for tiny sets.
When does a microservice split hurt?	When services share a DB or deploy together — distributed-systems cost, none of the autonomy.
Idempotency key placement?	At the command boundary, checked before side effects, stored with the result so retries return the original outcome.

Smart questions to ask the panel (signals seniority)

- "On-call culture — blameless postmortems, and who owns MTTR?"
- "How is this lead's technical authority drawn vs. the EM's people authority?"
- "Biggest architectural regret right now — and appetite to pay it down?"
- "How do you decide platform/shared vs. squad-owned concerns?"
- "Current change-failure rate and lead time — and the one-year target?"

7. One-Page Cheat Sheet (print & carry)

.NET / C#

- Async frees the thread at I/O; no sync-context in Core.
- Thread-pool starvation = sync-over-async (`.Result`). Go async end-to-end.
- `ValueTask` only on hot, sync-completing paths; await once.
- LOH >85KB → pool buffers; Server GC for throughput.
- `Span<T>` for transient slicing; no boxing in hot loops.
- Middleware order: exception → correlation → authN → authZ → tenant → endpoint.

SQL Server

- Diagnose from the *actual* plan + Query Store, not a rewrite.
- Fast→slow = stale stats or parameter sniffing.
- Deadlock: canonical lock order + covering index; retry is defense only.
- Composite index = Equality, Range, Include (covering kills key lookups).
- Optimistic (`rowversion`) default; pessimistic only for tiny critical sections.

React

- Separate server-state (React Query) from client-state (RTK/Zustand).
- Context = DI, not a store; split providers.
- Fix re-renders with Profiler + stable refs + virtualization, not blanket `useMemo` .
- Tokens in `HttpOnly` cookie / BFF; never localStorage.

Cloud / Architecture

- Exactly-once effect = at-least-once + idempotent consumers + outbox.
- Service Bus (ordered commands) vs Event Hubs (high-volume events).
- Tenancy: silo (isolation) / pool (density + RLS) / bridge; hybrid is fine.
- Zero-downtime: Blue/Green slots + Canary, expand/contract schema.
- Zero Trust: verify explicitly, least privilege, assume breach; identity is the boundary.
- Resilience: Polly timeout + retry-jitter + circuit breaker + bulkhead.

Leadership (STAR spine)

- Scope creep → capacity map + thin slice + options with trade-offs.
- Release block → impact/options/recommendation/ask; shield the team.
- Architecture conflict → ADR + CI architecture tests depersonalize it.
- Always close with a metric + "what I'd do differently."

SRE / Delivery

- Pyramid: Unit → Integration (Testcontainers) → thin E2E (Playwright).
- SLO + error budget drives ship-vs-freeze.
- OTel tracing cuts MTTR; DORA metrics track the delivery system.
- Auto-rollback on SLI breach; every change flagged + reversible.

AI / Cursor

- Force multiplier, accountable for every merged line.
- Composer refactors + `.cursorrules` conventions + reviewed AI tests.
- Audit diffs for leaks/sync-over-async/SQL-injection/secrets.

Prepared as a strict hiring-panel coaching artifact. Calibrate specifics (team sizes, metrics, incident numbers) to your real history — the frameworks are scaffolding; your authentic stories win the offer.